



Relatório - Solução do Problema do Caixeiro Viajante (TSP) Euclidiano

Henrique dos Reis {hdreis@inf.ufpel.edu.br}, Luísa Carvalho Böhm {lcbohm@inf.ufpel.edu.br}, Murilo Costa Salem {mcsalem@inf.ufpel.edu.br}

1. INTRODUÇÃO:

O Problema do Caixeiro Viajante (TSP) consiste em encontrar uma rota que visite cada cidade uma única vez e retorne ao ponto de partida, minimizando a soma dos custos das arestas percorridas. Apesar de sua formulação simples, o número de rotas possíveis cresce rapidamente com o aumento da quantidade de cidades, tornando a busca pela melhor solução cada vez mais difícil em instâncias maiores.

Diante dessa dificuldade, diferentes estratégias podem ser utilizadas. Algoritmos heurísticos buscam produzir boas soluções com baixo custo computacional, sem necessariamente garantir o resultado ótimo. Já abordagens de busca exata procuram explorar o espaço de soluções de forma sistemática, utilizando critérios de poda para evitar a avaliação de caminhos que não possam melhorar a melhor solução conhecida.

Neste trabalho, foram avaliadas cinco matrizes de adjacência, contendo 6, 11, 15, 29 e 44 cidades. Foram comparadas duas abordagens: a heurística do Vizinho Mais Próximo (VMP), utilizada para obter soluções de forma rápida, e uma implementação de Branch and Bound (B&B), proposta como abordagem exata.

A comparação considera tanto a qualidade das rotas encontradas quanto o custo computacional de cada método. Para isso, são analisados os custos das soluções, o tempo de execução e o comportamento das abordagens conforme o tamanho das instâncias aumenta. Conforme definido no enunciado, os valores presentes nos nomes dos arquivos são utilizados como ótimos de referência para o cálculo do gap das soluções obtidas.

2. METODOLOGIA:

2.1. Heurística: Vizinho Mais Próximo

O programa `tsp_aprox.cpp` implementa a heurística do Vizinho Mais Próximo (VMP), utilizando a cidade 0 como ponto inicial. A partir dela, o algoritmo constrói a rota de maneira incremental. Em cada etapa, percorre a linha da matriz de adjacência correspondente à cidade atual e seleciona, entre as cidades ainda não visitadas, aquela cuja aresta possui o menor custo. Quando duas ou mais cidades apresentam o mesmo custo mínimo, o desempate é realizado pelo menor índice.

A cidade escolhida é então marcada como visitada, adicionada à rota e passa a ser o novo ponto de referência para a próxima decisão. Esse procedimento é repetido até que todas as cidades tenham sido visitadas exatamente uma vez. Ao final, acrescenta-se a aresta que liga a última cidade à cidade 0,

fechando o ciclo, e o programa apresenta a sequência de cidades percorridas juntamente com o custo total da rota.

A implementação utiliza uma matriz de inteiros para representar os custos entre as cidades, um vetor para registrar quais cidades já foram visitadas e outro vetor para armazenar a rota construída. Como, para cada cidade adicionada ao percurso, pode ser necessário examinar até n possíveis destinos, a construção da solução apresenta complexidade de tempo $O(n^2)$. A memória auxiliar utilizada pelos vetores é $O(n)$, além dos $O(n^2)$ necessários para armazenar a matriz de adjacência.

Por se tratar de uma estratégia gulosa, o VMP toma cada decisão considerando apenas o menor custo disponível naquele momento, sem avaliar seus efeitos sobre as escolhas seguintes. Essa característica torna o método simples e rápido, mas não oferece garantia de obtenção da solução ótima. Uma escolha local de baixo custo pode, por exemplo, restringir as alternativas posteriores e fazer com que arestas mais caras precisem ser utilizadas no final do percurso. Nesta implementação, não foram empregadas estratégias adicionais de melhoria, como a execução a partir de diferentes cidades iniciais ou a aplicação de busca local sobre a rota obtida.

2.2. Abordagem proposta: Branch and Bound

O programa `tsp_exato.cpp` implementa uma estratégia de Branch and Bound (*B&B*) para explorar o espaço de possíveis rotas. Diferentemente da heurística do Vizinho Mais Próximo, que constrói apenas uma rota por meio de decisões locais, o B&B mantém simultaneamente diferentes rotas parciais e decide quais delas devem continuar sendo exploradas. Cada estado da busca armazena o caminho construído até o momento, o conjunto de cidades já visitadas, o custo acumulado da rota parcial e uma estimativa denominada *bound*.

Os estados são organizados em uma fila de prioridade, que seleciona primeiro aquele com o menor valor de *bound*. A partir do estado escolhido, novos estados são gerados adicionando-se, uma a uma, as cidades ainda não visitadas ao caminho atual. Quando um estado contém todas as cidades, adiciona-se o retorno à origem e obtém-se uma rota completa. Caso seu custo seja menor que o da melhor solução conhecida até então, esse valor passa a ser utilizado como novo incumbente da busca.

O *bound* tem a função de estimar o menor custo que uma determinada rota parcial ainda poderia alcançar. Na raiz da busca, essa estimativa é calculada a partir das duas menores arestas incidentes em cada cidade: esses valores são somados, divididos por dois e arredondados para cima. Nos estados descendentes, o programa atualiza incrementalmente essa estimativa, removendo parcelas associadas aos menores custos previamente considerados e acrescentando o custo da nova aresta inserida no caminho.

A principal vantagem desse mecanismo é permitir a poda de partes do espaço de busca. Se o *bound* de um estado indicar que nenhuma continuação daquele caminho poderia produzir uma solução melhor que o incumbente atual, esse estado pode ser descartado sem que seus descendentes sejam explorados. Dessa forma, o algoritmo procura evitar a enumeração completa de todas as permutações possíveis. Entretanto, para que essa poda preserve a exatidão do método, o *bound* precisa constituir um limite inferior válido para qualquer solução que possa ser obtida a partir daquele estado.

Mesmo com podas, o pior caso continua apresentando crescimento combinatório, pois a quantidade de rotas parciais pode aumentar rapidamente conforme o número de cidades cresce. Além do custo de processamento necessário para expandir esses estados, a utilização de uma fila de prioridade pode exigir grande quantidade de memória, já que muitos estados podem permanecer

armazenados simultaneamente aguardando exploração. Esse comportamento é particularmente relevante na análise experimental das instâncias maiores.

2.3. Protocolo experimental

Os arquivos-fonte foram compilados com G++ 13.3.0, utilizando o padrão C++17 e a opção de otimização -O2, em ambiente Linux. Os experimentos foram executados em um computador com processador Intel Core Ultra 9 285K, 24 CPUs lógicas e aproximadamente 62 GiB de memória RAM.

Cada instância foi executada separadamente pelos algoritmos exato e aproximativo. O algoritmo exato, baseado em Branch and Bound, recebeu um limite interno de 60 segundos por instância. Como salvaguarda, também foi utilizado o comando externo timeout, com envio de SIGTERM após 60 segundos e tolerância adicional de 2 segundos antes de um encerramento forçado. Não foi imposto limite específico de memória.

As instâncias tsp1_253.txt, tsp2_1248.txt e tsp3_1194.txt foram concluídas pelo algoritmo exato dentro do prazo. As instâncias maiores, tsp4_7013.txt e tsp5_27603.txt, excederam o limite de 60 segundos e foram interrompidas. Para essas instâncias, foram considerados somente o tempo e o custo do algoritmo aproximativo.

Os tempos registrados correspondem às medições internas realizadas pelos próprios programas durante a etapa de solução. Portanto, não incluem a criação do processo, a leitura do arquivo de entrada nem a finalização do programa. Como algumas execuções aproximativas terminaram em poucos microssegundos, essas medições estão sujeitas à resolução do relógio e a pequenas variações causadas pelo sistema operacional.

3. RESULTADOS DAS EXECUÇÕES DO TSP:

O algoritmo exato foi limitado a 60 segundos por instância. O valor ótimo de referência foi extraído do nome de cada arquivo. O gap percentual foi calculado pela seguinte fórmula:

$$\text{gap (\%)} = [(C_{\text{solução}} - C_{\text{ótimo}}) / C_{\text{ótimo}}] \times 100$$

Instância	Cidades	Ótimo	Custo VMP	Gap VMP	Custo B&B
TSP1	11	253	299	18,18%	254
TSP2	6	1248	1272	1,92%	1248
TSP3	15	1194	1260	5,53%	1194
TSP4	44	7013	10587	50,96%	—
TSP5	29	27603	36399	31,87%	—

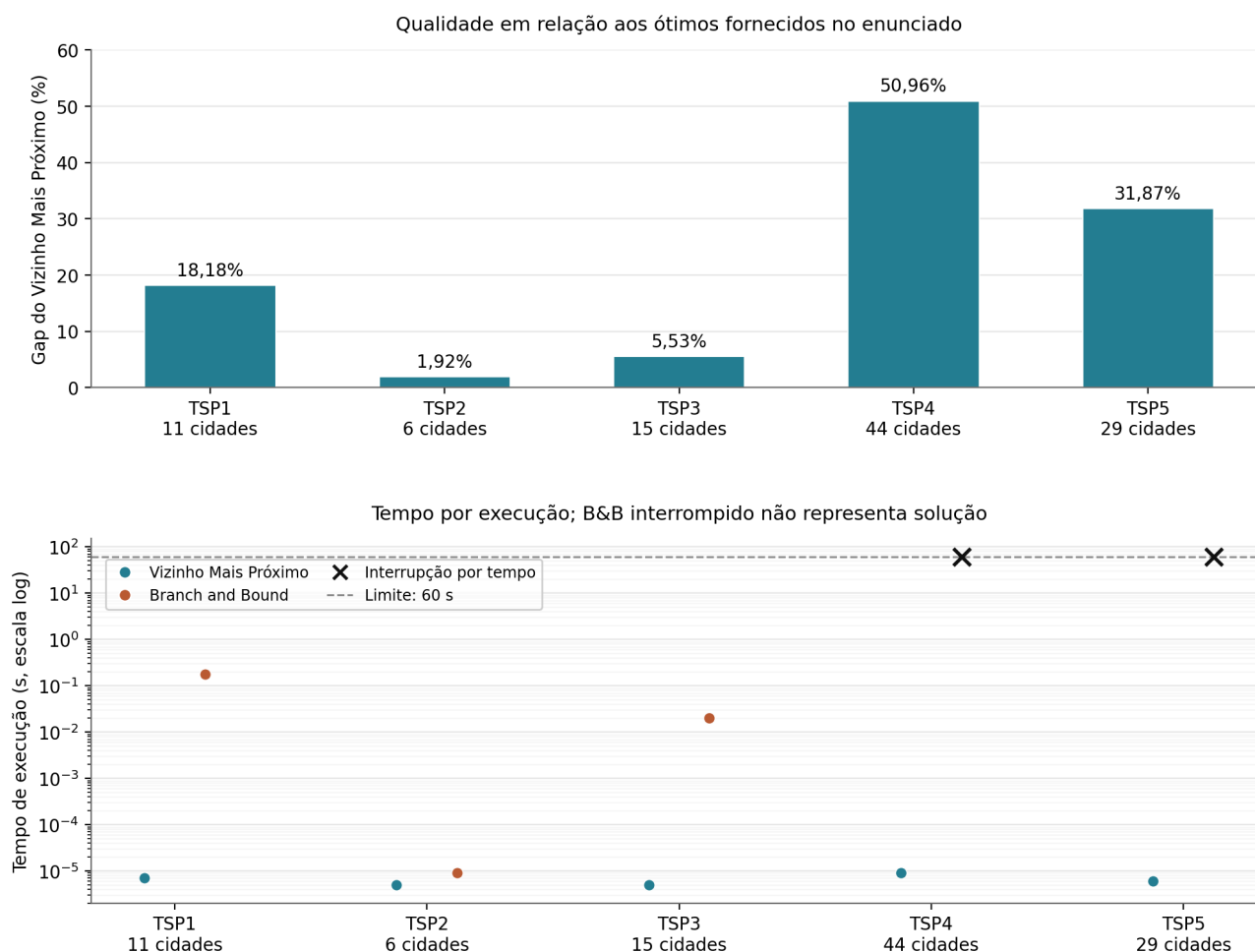
Tabela 1 — Custo e gap da heurística; resultado final da busca B&B.

As execuções exatas de tsp4_7013.txt (44 cidades) e tsp5_27603.txt (29 cidades) excederam o limite de 60 segundos e foram encerradas. Conforme solicitado, para essas instâncias são relatados abaixo somente os resultados do algoritmo aproximativo.

Observação: embora tenha terminado normalmente, o programa exato retornou custo 254 para *tsp1_253.txt*, uma unidade acima do ótimo 253 informado no nome do arquivo. O valor da tabela é o resultado efetivamente produzido pelo programa, sem qualquer alteração no código.

Instância	Tempo VMP (s)	Tempo B&B (s)	Status B&B
TSP1	0,000007	0,172889	Terminou;
TSP2	0,000005	0,000010	Terminou; ótimo confirmado
TSP3	0,000006	0,019320	Terminou; ótimo confirmado
TSP4	0,000008	—	Interrompido por timeout
TSP5	0,000007	—	Interrompido por timeout

3.1. Tempo computacional versus qualidade



O Vizinho Mais Próximo (VMP) produziu uma rota completa para todas as instâncias em tempos internos entre 0,000005 e 0,000008 segundo. A rapidez, entretanto, não implicou qualidade uniforme. O menor gap foi observado em TSP2, com 1,92%, enquanto o maior ocorreu em TSP4, com 50,96%. Os demais gaps foram 18,18% em TSP1, 5,53% em TSP3 e 31,87% em TSP5. Assim, embora a heurística forneça rapidamente uma resposta viável, a escolha gulosa da cidade mais próxima pode produzir decisões locais difíceis de compensar nas etapas seguintes.

Nas três instâncias em que terminou, o Branch and Bound (B&B) melhorou o custo obtido pelo VMP. Em TSP1, o custo caiu de 299 para 254; em TSP2, de 1272 para 1248; e, em TSP3, de 1260 para 1194. Apesar disso, somente os resultados de TSP2 e TSP3 coincidiram com os ótimos fornecidos e confirmados independentemente. Em TSP1, o B&B terminou com custo 254, ainda 0,40% acima do ótimo 253.

O tempo do B&B também não cresceu de maneira monotônica com o número de cidades. TSP1, com 11 cidades, levou 0,172889 segundo, enquanto TSP3, com 15 cidades, levou 0,019320 segundo. A primeira foi, portanto, aproximadamente nove vezes mais demorada, mesmo sendo menor. Esse comportamento mostra que o esforço da busca depende não apenas da dimensão da matriz, mas também da distribuição dos custos, da ordem em que os estados são explorados e da eficácia das podas.

Os tempos apresentados não são medianas nem tempos totais de parede. Houve uma execução registrada por algoritmo e instância, e a medição foi feita internamente pelos programas. Ela não inclui a criação do processo, a leitura do arquivo nem a impressão final. Por isso, especialmente nos valores de poucos microssegundos do VMP, pequenas diferenças não devem ser interpretadas como comparações precisas de desempenho.

3.2. Em que tamanho a busca deixou de ser viável?

Entre as instâncias avaliadas, o B&B não concluiu TSP5 (29 cidades) nem TSP4 (44 cidades), ambas interrompidas após o limite de 60 segundos. Como não houve limite de memória, esses casos devem ser classificados como timeout.

Esse resultado não define 29 cidades como um limite geral do B&B, pois depende da implementação, das instâncias, da máquina e do tempo máximo adotado. Além disso, como as execuções foram interrompidas antes da saída final, não foi obtida uma rota verificável para TSP4 e TSP5; assim, os 60 segundos indicam apenas o momento da interrupção da busca.

4. CONCLUSÃO:

O experimento evidencia o compromisso entre rapidez e qualidade. O VMP produziu rotas válidas para todas as instâncias em poucos microssegundos de tempo interno, mas apresentou gaps entre 1,92% e 50,96% em relação aos ótimos fornecidos. O B&B encontrou os valores ótimos de TSP2 e TSP3, mas retornou uma solução subótima em TSP1 e não concluiu TSP4 nem TSP5 dentro de 60 segundos.

Os resultados também mostram que concluir uma execução não equivale, por si só, a comprovar o ótimo. O limite inferior do B&B precisa ser revisto, como demonstra o prefixo da rota ótima de TSP1. Também é necessário uniformizar o significado das entradas de custo zero para que os dois algoritmos resolvam o mesmo problema. Como melhorias futuras, seria útil fornecer ao B&B uma solução inicial heurística, preservar e exportar a melhor rota quando houver interrupção e registrar consumo de memória e estados explorados. Para o VMP, múltiplas cidades de partida e uma etapa de busca local, podem reduzir os gaps observados.